

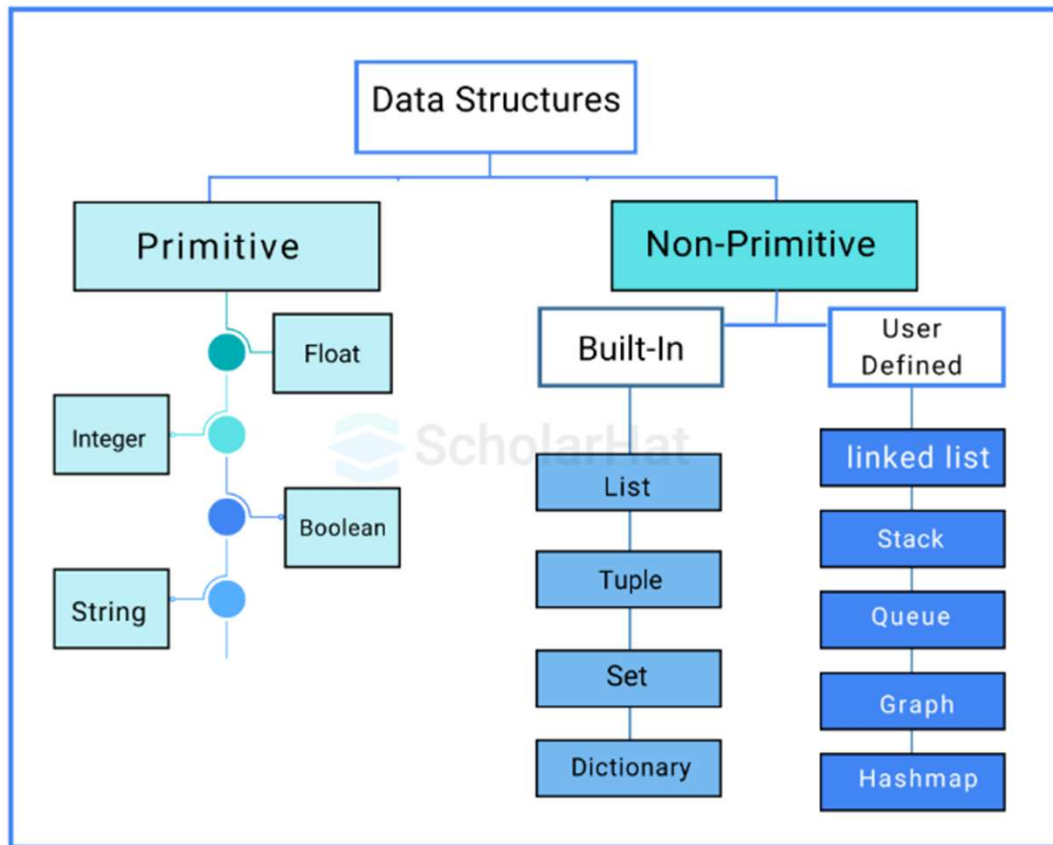
Introduction to Computer Applications

Lecture 4: Data Structures

Data Structures

- Data structures are a way of organizing and storing data so that they can be accessed and worked with efficiently.
- They define the relationship between the data, and the operations that can be performed on the data.
- Two main types
 - Primitive Data Structures/ Types
 - Store a single value of a specific type
 - Integer , Float, String, Boolean
 - Non-Primitive Data Structures
 - Store multiple values of different types in different structures
 - Arrays , Lists, Tuples, Sets , Dictionaries
- Abstract Data Types
 - Defined with the help of classes and Object Oriented Programming Techniques
 - Distance , Time , Point in Cartesian Plane etc.

Python Data Structures



Lists in Python

List

- A list is a sequence type in Python (other seq types : tuples, strings)
- Sequence
 - A sequence is a positionally ordered collection of items
 - You can refer any item with the help of an index like a[0] , a[1] etc.
 - Index starts at 0
 - For a sequence with 10 elements , index will range in 0-9 .
 - List , tuples and strings are python sequences
- Mutable
 - List elements can be changed
- Can contain heterogeneous data elements
 - Can store strings, integers , floats , etc.

Python Lists – Example

List in Python

`L = [ 20, 'Jessa', 35.75, [30, 60, 90] ]`

↑ ↑ ↑ ↑

`L[0]` `L[1]` `L[2]` `L[3]`

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Changeable:** List is mutable and we can modify items.
- ✓ **Heterogeneous:** List can contain data of different types
- ✓ **Contains duplicate:** Allows duplicates data

List Operations

- **Creating list**

- Creating an empty list
 - `mylist=[]`
- Creating a list with some values
 - `languages=['C', 'C++', 'java', 'python', 'golang']`
- Converting an iterable to a list
 - `numbers=list(range(0,20))`
- Using repetition operator
 - * is multiplication operator . But if operand on the left is a seq type like list , and operand on the right is an integer , then * acts like a repetition operator and makes multiple copies of the list and joins them altogether.
 - `list2=[10]*5`

range() – a useful built-in function

- range()
 - Creates a type of object called an iterable
- iterable
 - An object that contains a sequence of values that can be iterated over
 - Used in counter-controlled loops (for loop for example)
- Syntax
 - range(start, stop, step)
 - By default , **start**=0 . It is an optional argument
 - **stop** is required . Specifies the end point. Itself not included
 - **step** specifies the increment. By default, it is 1

range() – syntax

How to use Python range(start, stop, step)

range() returns the immutable sequence of numbers starting from the given **start integer to the stop-step**. Each number is incremented by adding step value to its preceding number

range(0, 6, 1) →

0	1	2	3	4	5
---	---	---	---	---	---

Step. (Optional) Specify the increment.
Default is 1

Stop. (Required) specifying at which
position to stop. Not part of the result

Start. (Optional) Start number of
sequence. Default is 0

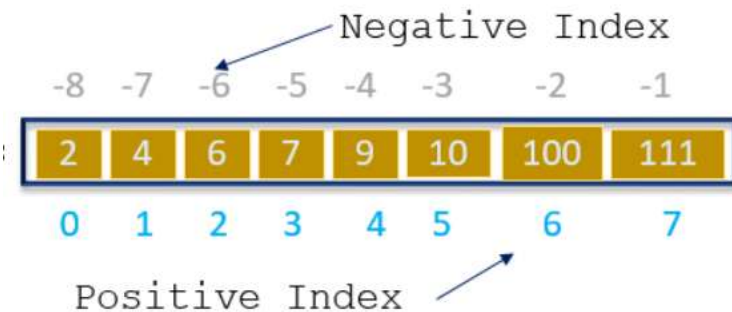
```
for i in range(6):  
    print(i)
```

It returns a range object not list
`type(range(6)) -> class 'range'`

List Operations

- **Accessing List Elements**

- Individual list elements can be accessed by specifying the index
- List index starts at 0
- There are two types of index
 - Positive index
 - Starts from 0 , and traverse from left to right
 - Negative Index
 - Starts form -1 and traverse from right to left



- **Mutation**

- You can change the list values
- `mylist[4]=8` for example

Some useful built-in list methods

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

List Operators

- Concatenation Operator (+)
 - Concatenates two or more strings together
- in operator
 - Checks whether an element is present in a list
- not in operator
 - Checks whether an element is not present in list
- * operator
 - Acts as a replication operator

Tuples

Tuples

- A tuple is an immutable sequence type
- The primary difference between list and tuples is mutability/immutability
 - Once a tuple is created it cannot be changed

Tuples in Python

T = (20, 'Jessa', 35.75, [30, 60, 90])

↑ ↑ ↑ ↑

T[0] T[1] T[2] T[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Unchangeable:** Tuples are immutable and we can't modify items.
- ✓ **Heterogeneous:** Tuples can contains data of types
- ✓ **Contains duplicate:** Allows duplicates data

List in Python

L = [20, 'Jessa', 35.75, [30, 60, 90]]

↑ ↑ ↑ ↑

L[0] L[1] L[2] L[3]

- ✓ **Ordered:** Maintain the order of the data insertion.
- ✓ **Changeable:** List is mutable and we can modify items.
- ✓ **Heterogeneous:** List can contain data of different types
- ✓ **Contains duplicate:** Allows duplicates data

Tuples vs Lists

- Tuples support all of the same operations as lists except those which change the contents of the list
- Tuples support the following:
 - Subscript indexing (for retrieving element values only)
 - Methods such as index()
 - Built-in function like len , min and max
 - Slicing Expressions
 - The in operator
 - The + and * operators
- Tuples do not support methods like
 - append(), remove() , insert() , reverse() and sort()

Why tuples exist?

- Immutability is the main difference between tuples and lists
- Is immutability so important?
- Processing tuples is faster than processing lists
 - When you are processing lots of data and your data will not be modified
- Tuples are safe
 - You store data once and rest assured that your data will not be modified
 - Accidentally or otherwise by any program
- There are other operations in Python that require the use of tuple
 - As you will learn more about Python , you will encounter tuples more frequently

Sets

Sets in Python

- A set is a built-in container type data structure in Python
 - An unordered collection of data
 - You cannot access elements with the help of index
 - You cannot insert elements by using index
 - mutable
 - Set elements can be modified in place
 - Heterogenous
 - You can store elements with different primitive and non-primitive types
 - Unique
 - Set elements are unique
 - Elements are enclosed in {} brackets

Sets – Example

Set in Python

$S = \{ 20, 'Jessa', 35.75 \}$

- ✓ **Unordered:** Set doesn't maintain the order of the data insertion.
- ✓ **changeable:** Sets are mutable
- ✓ **Heterogeneous:** Set can contains data of all types
- ✓ **Unique:** Set doesn't allows duplicates items

Creating Sets

```
1 # create a set using {}
2 # set of mixed types intger, string, and floats
3 sample_set = {'Mark', 'Jessa', 25, 75.25}
4 print(sample_set)
5 # Output {25, 'Mark', 75.25, 'Jessa'}
6
7 # create a set using set constructor
8 # set of strings
9 book_set = set(("Harry Potter", "Angels and Demons", "Atlas Shrugged"))
10 print(book_set)
11 # output {'Harry Potter', 'Atlas Shrugged', 'Angels and Demons'}
12
13 print(type(book_set))
14 # Output class 'set'
```

Creating a set from Lists

```
# list with duplicate items
number_list = [20, 30, 20, 30, 50, 30]
# create a set from a list
sample_set = set(number_list)

print(sample_set)
# Output {50, 20, 30}
```

Creating Empty Sets

When we don't pass any item to the set constructor then it will create an **empty set**.

```
empty_set = set()
print(type(empty_set))
# class 'set'
```

When the same object is created without any items inside the curly brackets then it will be created as a **dictionary** which is another built-in data structure in Python.

So whenever you wanted to create an empty set always use the `set()` constructor.

```
emptySet = {}
print(type(emptySet)) # class 'dict'
```

Accessing items in a set

The items of the set are unordered and they don't have any index number. In order to access the items of a set, we need to iterate through the set object using a `for` loop

```
book_set = {"Harry Potter", "Angels and Demons", "Atlas Shrugged"}  
for book in book_set:  
    print(book)
```

Output:

```
Angels and Demons  
Atlas Shrugged  
Harry Potter
```

Checking if an element is present in a set

The `in` operator checks whether the item is present in the set, and returns `True` if it present otherwise, it will return `False`.

```
book_set = {"Harry Potter", "Angels and Demons", "Atlas Shrugged"}
if 'Harry Potter' in book_set:
    print("Book exists in the book set")
else:
    print("Book doesn't exist in the book set")
# Output Book exists in the book set

# check another item which is not present inside a set
print("A Man called Ove" in book_set)
# Output False
```

Adding items to a set

Though the value of the item in a Set can't be modified. We can add new items to the set using the following two ways.

1. The `add()` method: The `add()` method is used to add one item to the set.
2. Using `update()` Method: The `update()` method is used to multiple items to the Set. We need to pass the list of items to the `update()` method

Example

```
book_set = {'Harry Potter', 'Angels and Demons'}
# add() method
book_set.add('The God of Small Things')
# display the updated set
print(book_set)
# Output {'Harry Potter', 'The God of Small Things', 'Angels and Demons'}

# update() method to add more than one item
book_set.update(['Atlas Shrugged', 'Ulysses'])
# display the updated set
print(book_set)
# Output {'The God of Small Things', 'Angels and Demons', 'Atlas Shrugged', 'Harry Pott
```

Removing items from a set

Method	Description
<code>remove()</code>	To remove a single item from a set. This method will take one parameter, which is the item to be removed from the set. Throws a <code>keyerror</code> if an item not present in the original set
<code>discard()</code>	To remove a single item that may or may not be present in the set. This method also takes one parameter, which is the item to be removed. If that item is present, it will remove it. It will not throw any error if it is not present.
<code>pop()</code>	To remove any random item from a set
<code>clear()</code>	To remove all items from the Set. The output will be an empty set
<code>del set</code>	Delete the entire set

Python Set methods to remove items

Removing Items from a set

```
color_set = {'red', 'orange', 'yellow', 'white', 'black'}

# remove single item
color_set.remove('yellow')
print(color_set)
# Output {'red', 'orange', 'white', 'black'}
```

```
# remove any random item from a set
deleted_item = color_set.pop()
print(deleted_item)

# remove all items
color_set.clear()
print(color_set)
# output set()

# delete a set
del color_set
```

Set Operations

Operation	Definition	Operator	Method
Union	All the items of both Sets will be returned. Only the duplicate items will be dropped.		<code>union()</code>
Intersection	Only the items common in both sets will be returned.	&	<code>intersection()</code>
Difference	Return the unique elements in the first set which is not in the second set.	-	<code>difference()</code>
Symmetric Difference	Return the elements of both sets which is not common.	^	<code>symmetric_difference()</code>

Copying a Set

In Python, we can copy the items from one set to another in three ways.

- Using `copy()` method.
- Using the `set()` constructor
- Using the `=` (assignment) operator (assigning one set to another)

The difference is while using the `=` (assignment) operator any modifications we make in the original set will be reflected in the new set. But while using the `copy()` method, the new set will not reflect the original set's changes.

Frozen Set

- Although a set is a mutable data structure, it can be made immutable if required
 - When we need a set to be readable only
 - Elements can neither be added nor removed
- Create a frozen set

```
rainbow = ('violet', 'indigo', 'blue')
f_set = frozenset(rainbow)
# Add to frozenset
f_set.add(f_set)
# output AttributeError: 'frozenset' object has no attribute 'add'
```

- All set operations which do not modify the set contents are allowed.

Dictionaries

Dictionary

- A built-in data structure that stores values in the form of a key-value pairs
- Values can be retrieved by using the key

Unordered collections of unique values stored in (Key-Value) pairs.

```
d = {'a': 10, 'b': 20, 'c': 30}
```


d['a'] d['b'] d['c']

- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

Creating a Dictionary

- Different Options:

- Using { } braces to define a dictionary with some key-value pairs

```
# create a dictionary using {}  
person = {"name": "Jessa", "country": "USA", "telephone": 1178}
```

- Creating a dictionary from a sequence

```
# create a dictionary from sequence having each item as a pair  
person = dict([("name", "Mark"), ("country", "USA"), ("telephone", 1178)])
```

- Creating a dictionary with values as a list

```
# create dictionary with value as a list  
person = {"name": "Jessa", "telephones": [1178, 2563, 4569]}
```

- Creating an empty dictionary

```
emptydict = {}
```

Accessing Dictionary Elements

Use the following dictionary methods to retrieve all key and values at once

Method	Description
<code>keys()</code>	Returns the list of all keys present in the dictionary.
<code>values()</code>	Returns the list of all values present in the dictionary
<code>items()</code>	Returns all the items present in the dictionary. Each item will be inside a tuple as a key-value pair.

Accessing Dictionary Elements

- Two different ways:
 - Using key inside []
 - Using key as an argument to get()

```
# create a dictionary named person
person = {"name": "Jessa", "country": "USA", "telephone": 1178}

# access value using key name in []
print(person['name'])
# Output 'Jessa'

# get key value using key name in get()
print(person.get('telephone'))
# Output 1178
```

Getting all keys and all values

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}

# Get all keys
print(person.keys())
# output dict_keys(['name', 'country', 'telephone'])
print(type(person.keys()))
# Output class 'dict_keys'

# Get all values
print(person.values())
# output dict_values(['Jessa', 'USA', 1178])
print(type(person.values()))
# Output class 'dict_values'

# Get all key-value pair
print(person.items())
# output dict_items([('name', 'Jessa'), ('country', 'USA'), ('telephone', 1178)])
print(type(person.items()))
# Output class 'dict_items'
```

Iterating a Dictionary / Accessing Elements

- We can iterate through a dictionary using a for loop to access keys and associated values

```
person = {"name": "Jessa", "country": "USA", "telephone": 1178}

# Iterating the dictionary using for-loop
print('key', ':', 'value')
for key in person:
    print(key, ':', person[key])
```

- We can also use items()
 - In each item index [0] contains key and index [1] contains values

```
# using items() method
print('key', ':', 'value')
for key_value in person.items():
    # first is key, and second is value
    print(key_value[0], key_value[1])
```

Adding / Modifying Elements to a Dictionary

- There are two ways:
 - Assign a value to a key like Age["Ali"]=30
 - Use update method and pass a key-value
 - Age. Update ({"Ali":30})
 - Can be used to add multiple key-value pairs
 - Example

```
person = {"name": "Jessa", 'country': "USA", "telephone": 1178}  
# update dictionary by adding 2 new keys  
person["weight"] = 50  
person.update({"height": 6})
```

```
# pass new keys as dict  
person.update({"weight": 50, "height": 6})
```

```
# pass new keys as as list of tuple  
person.update([("city", "Texas"), ("company", "Google"),])
```

- Same process (assignment and update()) can be used to update a value against a key

Removing / Deleting Items from a Dictionary

Method	Description
<code>pop(key)</code>	Return and removes the item with the <code>key</code> and return its value. If the <code>key</code> is not found, it raises <code>KeyError</code> .
<code>popitem()</code>	Return and removes the last inserted item from the dictionary. If the dictionary is empty, it raises <code>KeyError</code> .
<code>del key</code>	The <code>del</code> keyword will delete the item with the key that is passed
<code>clear()</code>	Removes all items from the dictionary. Empty the dictionary
<code>del dict_name</code>	Delete the entire dictionary

Other Dictionary Operations /Methods

- **len()** returns the number of key-value pairs
- **in** operator can be used to check key or value membership
- **sorted()** can be used to sort dictionary keys or values
- **update()** method can also be used to combine two dictionaries
 - **dict1.update(dict2)**
 - Duplicate will be removed
 - If duplicate occurs, dict2 values will be retained
- ****kwargs** can be used to unpack a dictionary and add its elements to another dictionary

```
student_dict1 = {'Aadya': 1, 'Arul': 2, }  
student_dict2 = {'Harry': 5, 'Olivia': 6}  
student_dict3 = {'Nancy': 7, 'Perry': 9}  
  
# join three dictionaries  
student_dict = {**student_dict1, **student_dict2, **student_dict3}
```

Copying a Dictionary

- Use `copy()` method
 - It creates a new dictionary with the same key-value pairs
 - `dict2=dict1.copy()`
- Use assignment `'='` operator
 - `dict2=dict1`
 - Copies the reference
 - Both `dict2` and `dict1` refer to the same object
 - Any changes made in `dict1` will reflect in `dict1` and vice versa

Nested Dictionaries

- A dictionary can be nested within another dictionary

```
# address dictionary to store person address
address = {"state": "Texas", "city": "Houston"}

# dictionary to store person details with address as a nested dictionary
person = {'name': 'Jessa', 'company': 'Google', 'address': address}

# Display dictionary
print("person:", person)
```

```
person: {'name': 'Jessa', 'company': 'Google', 'address': {'state': 'Texas', 'city': 'Houston'}}
```

```
# Get nested dictionary key 'city'
print("City:", person['address']['city'])
```

```
City: Houston
```

Iterating through Nested Dictionaries

```
person: {'name': 'Jessa', 'company': 'Google', 'address': {'state': 'Texas', 'city': 'Houston'}}
```

```
# Iterating outer dictionary
print("Person details")
for key, value in person.items():
    if key == 'address':
        # Iterating through nested dictionary
        print("Person Address")
        for nested_key, nested_value in value.items():
            print(nested_key, ': ', nested_value)
    else:
        print(key, ': ', value)
```

```
Person details
name : Jessa
company : Google
Person Address
state : Texas
city : Houston
```

Python Data Structures – A Comparison

List Vs Set Vs Dictionary Vs Tuple

Lists	Sets	Dictionaries	Tuples
List = [10, 12, 15]	Set = {1, 23, 34} Print(set) -> {1, 23, 24} Set = {1, 1} print(set) -> {1}	Dict = {"Ram": 26, "mary": 24}	Words = ("spam", "eggs") Or Words = "spam", "eggs"
Access: print(list[0])	Print(set). Set elements can't be indexed.	print(dict["ram"])	Print(words[0])
Can contains duplicate elements	Can't contain duplicate elements. Faster compared to lists	Can't contain duplicate keys, but can contain duplicate values	Can contains duplicate elements. Faster compared to lists
List[0] = 100	set.add(7)	Dict["Ram"] = 27	Words[0] = "care" -> TypeError
Mutable	Mutable	Mutable	Immutable - Values can't be changed once assigned
List = []	Set = set()	Dict = {}	Words = ()
Slicing can be done print(list[1:2]) -> [12]	Slicing: Not done.	Slicing: Not done	Slicing can also be done on tuples
<u>Usage:</u> Use lists if you have a collection of data that doesn't need random access. Use lists when you need a simple, iterable collection that is modified frequently.	<u>Usage:</u> - Membership testing and the elimination of duplicate entries. - when you need uniqueness for the elements.	<u>Usage:</u> - When you need a logical association b/w key:value pair. - when you need fast lookup for your data, based on a custom key. - when your data is being constantly modified.	<u>Usage:</u> Use tuples when your data cannot change. A tuple is used in combination with a dictionary, for example, a tuple might represent a key, because its immutable.

6/25/2016

Rajkumar Rampolla, Python

15



Institute of
Business Administration
Karachi

Leadership and Ideas for Tomorrow 51

Acknowledgement / Resources

The contents of this lecture is derived from the following resources:

- Arrays
 - <https://www.freecodecamp.org/news/python-array-tutorial-define-index-methods/>
 - <https://www.datacamp.com/tutorial/data-structures-python#abstract-data-type-and-data-structures>
- Lists & Tuples
 - Staring out with Python (Third Edition) By Tony Gaddis
 - Chapter 8 : List and Tuples
 - <https://pynative.com/python-range-function/>
 - <https://pynative.com/python-tuples/>
- Set and Dictionaries
 - Staring out with Python (Third Edition) By Tony Gaddis
 - Chapter 9 : Dictionaries and Sets
 - <https://pynative.com/python-sets/>
 - <https://pynative.com/python-dictionaries/>
 - <https://www.slideshare.net/rampalliraj/learn-python-for-beginners-part2>